SHA256 Specification

1. 功能介绍

该模块完成一个数据块的 SHA256 哈希摘要计算功能。软件配置好信息后(源数据地址，源数据 byte 长度)，它直接从源数据地址空间读取源数据块，做 SHA256 哈希摘要计算，把该段数据的 SHA256 值保存在 8 个内部配置寄存器中，SW 可以通过 APB 接口读取 SHA256 结果。软件配置寄存器采用 APB slave 接口，数据传输采用 AXI master 接口。

具体功能如下：

- 支持标准 SHA256 哈希算法；
- 最大源数据长度 4GB；
- 内嵌 DMA 模块，主动进行源数据读，最大 burst length 16；
- 每 66 个周期完成一个 512bit 数据块的处理；

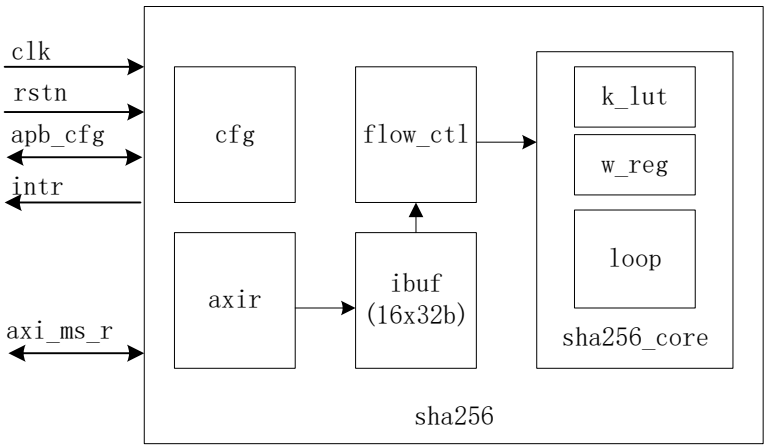
注：SHA256 协议规定的 padding 操作（有效数据尾补 1’b1，补 0x00，填写有效数据 bit 长度）由 SW 完成。

为了提高 AXI bus 的利用率，使用了如下 AXI 总线的特性：

- Read command outstanding，最多 outstanding 16x32bit 的数据；

本设计综合考虑了 AXI 总线 read burst 的 read latency 与 SHA256 数据吞吐率的关系，折衷选择了适合的内部 buffer 深度(目前是 16x32bit，缓存 1 个 512bit 数据块)。如果 AXI read latency 比较大(read latency 大于 64 个周期)，可以适当增加 input buffer 的深度。

设计框图如下图所示：



2. IO 功能定义

Name	Direction	Bits	Description
clk	I	1	时钟输入。
rstn	I	1	异步复位输入，低有效。
intr	O	1	中断输出，high level active。
axi_ms_inf	I/O	NA	标准的 AXI master 接口(32bits, maximal burst length is 16)，只有 read channel。
apb_cfg	I/O	NA	标准的 APB salve 接口。

AXI 接口时序：见 AMBA 标准协议：《AMBA™ AXI Protocal》，v1.0;
APB 接口时序：见 AMBA 标准协议：《AMBA™ 3 APB Protocal》，v1.0;

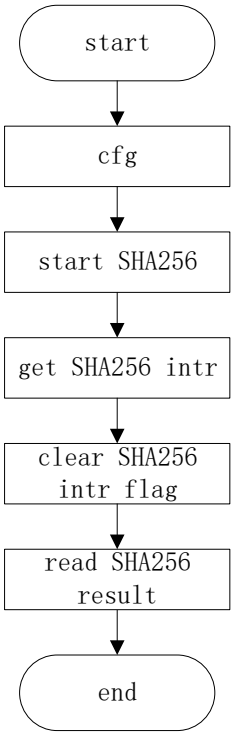
注：AMBA 协议是 ARM 公司定义的总线协议。

3. 寄存器定义

addr	bit	name	access	description
00	0	sha_sof	r/w	SW 写 1 启动一个数据块的 SHA256 处理。SW 读返回 0。
	31:1	NA	r	Reserved。
04	31:0	blk_base	r/w	SHA256 读取的源数据的系统地址，byte 单位。必须是 32bit 对齐的地址，即：blk_base[1:0] = 2'b00。
08	31:0	blk_len	r/w	SHA256 处理的数据块的大小，byte 单位，从 0 计数。必须是 64byte 的整数倍，即：blk_len[5:0] = 6'b111111。
0C	0	intr	r/w	1: SHA 结束一个数据块处理。SW 写 0 清除中断。
	31:1	NA	r	Reserved。
10	31:0	NA	r	Reserved。
14	31:0	NA	r	Reserved。
18	31:0	NA	r	Reserved。
1C	31:0	NA	r	Reserved。
20	31:0	hash_0	r	数据块的 hash256 结果的低 32bit,即：bit[0*32 +: 32]。
24	31:0	hash_1	r	数据块的 hash256 结果的 bit[1*32 +: 32]。
28	31:0	hash_2	r	数据块的 hash256 结果的 bit[2*32 +: 32]。

2C	31:0	hash_3	r	数据块的 hash256 结果的 bit[3*32 +: 32]。
30	31:0	hash_4	r	数据块的 hash256 结果的 bit[4*32 +: 32]。
34	31:0	hash_5	r	数据块的 hash256 结果的 bit[5*32 +: 32]。
38	31:0	hash_6	r	数据块的 hash256 结果的 bit[6*32 +: 32]。
3C	31:0	hash_7	r	数据块的 hash256 结果的最高 bit，即 bit[7*32 +: 32]。

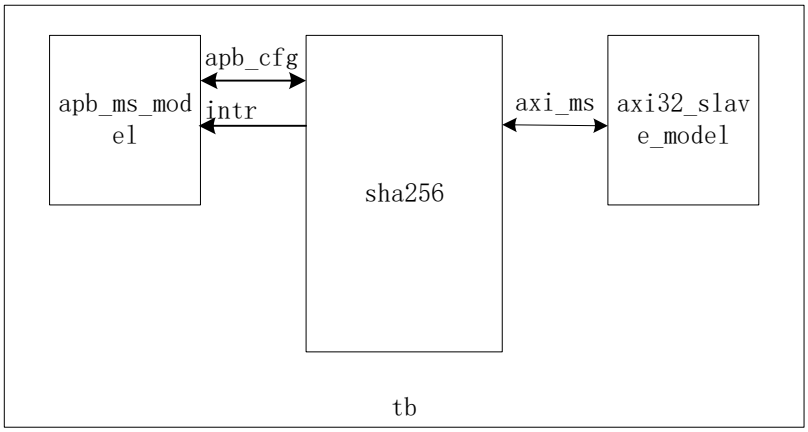
4. HW Control Flow



5. RTL Simulation

simulation 基本思路：由 c model 产生测试输入数据与对比结果，并保存在二进制文件中（见 testcase 目录的文件：inout_sha256.bin）。RTL 仿真时，TB 读取该文件，获取源数据信息并配置 SHA256 DUT，启动 DUT 开始一个数据块的 SHA256 计算。在一个数据块的 SHA256 哈希摘要计算结束后，用 RTL 输出与 C model 输出进行对比检查。

为了方便 debug，SHA256 哈希计算每次 loop 的中间结果也保存在文本文件中（见 testcase 目录的文件：sta_sha256.log）。

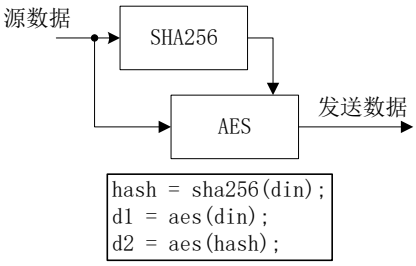


C model 输出的二进制文件的格式：

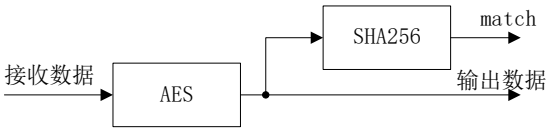
对比文件格式		
	内容	大小
header	test_bulk_num	4B
数据块 1	blk_len	4B
	in_data	blk_len B
	sha256	32B
数据块 2	blk_len	4B
	in_data	blk_len B
	sha256	32B
●●●		
数据块 N	blk_len	4B
	in_data	blk_len B
	sha256	32B

6. Improvement

- 减少处理一个 512bit block 需要的周期数（到 64 或 65 个周期）；
- 通过寄存器配置，SHA256 结果是否需要用 AXI write channel 写出；
- 如果每次 SHA256 哈希计算的数据块最长只有 512bit（含 padding 部分），考虑全 pipeline 结构增加数据吞吐率（比如一个周期处理一个 512bit 的哈希值）；
- 跟 AES 加密/解密配合，做成一个安全数据传输系统；




```
hash = sha256(din);
d1 = aes(din);
d2 = aes(hash);
```




```
din = aes(d1);
exp = aes(d2);
hash = sha256(din);
if(exp == hash)
   match = 1;
else
   match = 0;
```